Computer Organization & Assembly Language

Lecture / Lab-004

Leaf, Non-Leaf & Stack Pointer / Register in MIPS (QtSPIM)

Common Arithmetic Instructions (Registers Only)

Instruction	Meaning
`add rd, rs, rt`	Signed addition (overflow traps)
`addu rd, rs, rt`	Unsigned addition (wraparound)
`sub rd, rs, rt`	Subtraction
`mul rd, rs, rt`	Multiply (QtSPIM pseudo-instruction)
`div rs, rt`	Divide → quotient in **LO**, remainder in **HI**
`mflo rd`	Move quotient from LO
`mfhi rd`	Move remainder from HI
`sll rd, rs, shamt`	Shift left (× powers of 2)
`srl rd, rs, shamt`	Shift right logical (÷ powers of 2, nonnegative)
`sra rd, rs, shamt`	Shift right arithmetic (signed)

Memory Access Instructions

Instruction	Meaning
lw rt, label	Load word from memory
sw rt, label	Store word to memory

Syscall Registers (I/O Services)

$v0 value	Service
1	print integer
4	print string
10	exit program

MIPS = Load → Compute → Store
Registers are fast; memory is slow.

32 MIPS general-purpose registers

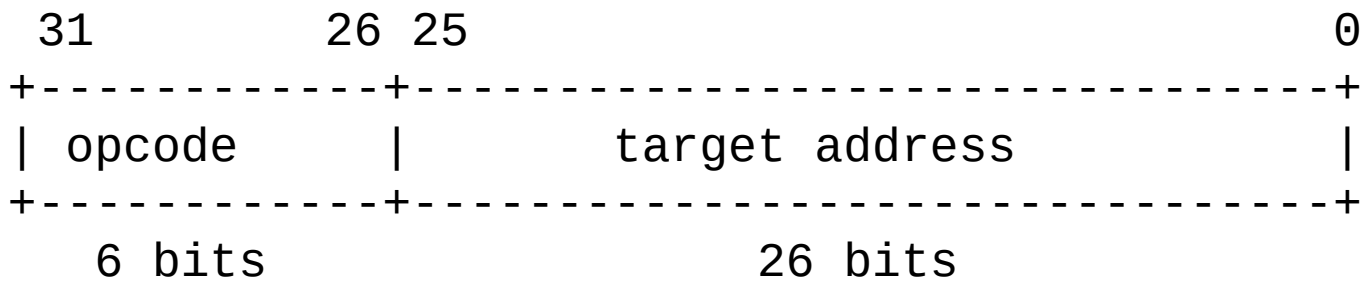
Group	Registers	Count	Typical use
Constant	$zero	1	always 0
Assembler reserved	$at	1	assembler temp
Return values	$v0–$v1	2	function results
Arguments	$a0–$a3	4	function params
Temporaries (caller-saved)	$t0–$t9	10	scratch values
Saved (callee-saved)	$s0–$s7 (+ $s8/$fp)	8 (+1)	preserved across calls
OS/kernel reserved	$k0–$k1	2	kernel use
Pointers/special	$gp, $sp, $fp/$s8, $ra	4	global/stack/frame/return

J-Type (Jump) Instructions

MIPS uses J-type instructions when execution needs to jump to another location in the program.

Instruction	Meaning	Instruction Type
j label	Jump to a target label	J-type
jal label	Jump to a target and save return address in $ra	J-type

- J-Type Instruction Format



- j → jump somewhere
- jal → jump somewhere AND remember where to return

Jump Instructions

- jal label
  - Jumps to function
  - Stores return address in $ra (PC + 4)
- jr $ra
  - Returns to caller using address in $ra

- Applying the Jal and Jr with $ra register to a division task of arithmetic division showing a leaf function/procedue.

```
jal divmod
...
jr $ra
```

Example Jump Instructions

Write a MIPS program that divides two integers using a leaf function named divmod. Pass the two integers to the function using $a0 and $a1. The function should return the quotient in $v0 and the remainder in $v1. Use jal to call the function and jr $ra to return to the caller. Print both the quotient and remainder.

```
.data
.text
.globl main
main:
   li $a0, 17
   li $a1, 5
   jal divmod
   move $a0, $v0
   li $v0, 1
   syscall

   li $a0, 10
   li $v0, 11
   syscall
```

```
move $a0, $v1
li $v0, 1
syscall

li $a0, 10
li $v0, 11
syscall

li $v0, 10
syscall

divmod:
div $a0, $a1
mflo $v0
mfhi $v1
jr $ra
```

17 ÷ 5
Quotient = 3
Remainder = 2

C++ Code

```
#include <iostream>
using namespace std;

int divmod(int a, int b)
{
    int quotient = a / b;
    int remainder = a % b;

    cout << "Remainder: " << remainder << endl;

    return quotient;
}

int main()
{
    int result = divmod(17, 5);

    cout << "Quotient: " << result << endl;

    return 0;
}
```

Leaf vs. Non-Leaf Function

- Leaf Function
  - Does NOT call another function
  - Does NOT need to save $ra
- Non-Leaf Function
  - Calls another function
  - MUST save and restore $ra
  - Uses stack pointer register $sp

Stack vs. Queue





Last-In-First-Out

First-In-First-Out

Stack Pointer ($sp)

- Holds address of top of stack
- Stack is memory used for temporary saves
- Stack grows downward (toward smaller addresses)
- Allocate space:
  - `addiu $sp, $sp, -4`
- Save value:
  - `sw $t0, 0($sp)`
- Restore value:
  - `lw $t1, 0($sp)`
- Deallocate space:
  - `addiu $sp, $sp, 4`

Stack Pointer ($sp)

- The stack grows in 4-byte steps when you're storing 32-bit words
  -4
  -8
  -12
  -16
  -20
  ...
- For example, suppose the original $sp = 1000:

```
addiu $sp, $sp, -4      # $sp = 996
addiu $sp, $sp, -4      # $sp = 992
addiu $sp, $sp, -4      # $sp = 988
addiu $sp, $sp, -4      # $sp = 984
```

  ↓ stack grows downward

- Example: if you know you need four words, you allocate all 16 bytes at once:

```
addiu $sp, $sp, -16     # allocate 16 bytes = 4 words
```

- Then use offsets within those 16 bytes:

```
sw $t0, 0($sp)          # first word
sw $t1, 4($sp)          # second word
sw $t2, 8($sp)          # third word
sw $ra, 12($sp)         # fourth word
```

STACK is LIFO (Last-In-First-Out)

- Push one item at a time

```
# Push $t0
addiu $sp, $sp, -4
sw    $t0, 0($sp)

# Push $t1
addiu $sp, $sp, -4
sw    $t1, 0($sp)

# Push $t2
addiu $sp, $sp, -4
sw    $t2, 0($sp)
```



The stack will look like this
Higher addresses

$t0 ← pushed FIRST
$t1
$t2 ← pushed LAST
↑
$sp

Lower addresses

However, using addiu $sp, $sp, -16 allocates an entire 16-byte stack frame at once. Once that frame exists, you can access any location within it using offsets:

```
sw $t0, 0($sp)
sw $t1, 4($sp)
sw $t2, 8($sp)
sw $ra, 12($sp)
```

Pop them individually, gives the reverse order:

```
lw    $t2, 0($sp)      # LAST IN → FIRST OUT
addiu $sp, $sp, 4

lw    $t1, 0($sp)
addiu $sp, $sp, 4

lw    $t0, 0($sp)      # FIRST IN → LAST OUT
addiu $sp, $sp, 4
```

No need to pop in Last in First out
You can pop (`lw`) any value in any order

Leaf vs. Non-Leaf Function

- Leaf Function
  - Does NOT call another function
  - Does NOT need to save $ra
- Non-Leaf Function
  - Calls another function
  - MUST save and restore $ra
  - Uses stack pointer register $sp

Non-Leaf divmod vs. Leaf divmod

```
divmod:
    addiu $sp, $sp, -4      # allocate stack space
    sw    $ra, 0($sp)       # save return address

    div   $a0, $a1
    mflo  $v0               # quotient
    mfhi  $v1               # remainder

    jal   print_newline     # call another function

    lw    $ra, 0($sp)       # restore return address
    addiu $sp, $sp, 4       # deallocate stack space
    jr    $ra
```

```
divmod:

    div   $a0, $a1
    mflo  $v0               # quotient
    mfhi  $v1               # remainder

    jr    $ra
```

Stack Pointer ($sp)

- It holds the address of the top of the current stack frame, and it changes when you allocate/free stack space.

```
.text
.globl main
main:
  li  $a0, 17
  li  $a1, 5
  jal divmod

  # print quotient ($v0)
  move $a0, $v0
  li   $v0, 1
  syscall

  # exit
  li   $v0, 10
  syscall
```

```
# leaf helper that prints newline
print_newline:
  li   $a0, 10
  li   $v0, 11
  syscall
  jr   $ra


# divmod is NOT leaf anymore
#(it calls print_newline),
# so it must save/restore $ra.
divmod:
  addi $sp, $sp, -4
  sw   $ra, 0($sp)
```

```
div  $a0, $a1
  mflo $v0
  mfhi $v1

# jal writes the return address into $ra
  jal  print_newline

  lw   $ra, 0($sp)
  addi $sp, $sp, 4
  jr   $ra
```

A leaf function is a function that does not call any other function

Stack Pointer ($sp) – Non Leaf Example

```
.data
newline: .asciiz "\n"

.text
.globl main

main:
    li $a0, 17
    li $a1, 5

    jal divmod

    # If divmod returns correctly, print 99
    li $a0, 99
    li $v0, 1
    syscall

    li $v0, 10
    syscall
```

Prints 99 to show that divmod returns correctly.
Commenting out the restore will print nothing

```
divmod:
    addiu $sp, $sp, -4
    sw    $ra, 0($sp)

    div   $a0, $a1
    mflo  $v0
    mfhi  $v1

    jal print_newline

    lw    $ra, 0($sp)       # RESTORE
    addiu $sp, $sp, 4

    jr $ra


print_newline:
    li $a0, 10
    li $v0, 11
    syscall

    jr $ra
```

Stack Pointer ($sp) – Non Leaf Example

```
.data
returned: .asciiz "Successfully returned to main!\n"
inside:   .asciiz "Inside helper function\n"

.text
.globl main

main:
    jal calculate

    # We only get here if calculate returns correctly
    li $v0, 4
    la $a0, returned
    syscall

    li $v0, 10
    syscall
```

print a message after the non-leaf function returns

```
#Non-leaf function
calculate:
    # Save return address to main
    addiu $sp, $sp, -4
    sw    $ra, 0($sp)

    # Call another function
    jal helper

    # Restore return address to main
    lw    $ra, 0($sp)
    addiu $sp, $sp, 4

    # Return to main
    jr $ra


#leaf function
helper:
    li $v0, 4
    la $a0, inside
    syscall

    jr $ra
```

In Class Task

```
#include <iostream>
using namespace std;

// Leaf function
int doubleValue(int x)
{
    return x * 2;
}

// Non-leaf function
int calculate(int a, int b)
{
    int sum = a + b;

    // calculate calls another function
    int result = doubleValue(sum);

    return result;
}

int main()
{
    int result = calculate(8, 4);

    cout << result << endl;

    return 0;
}
```

C++	MIPS
calculate(8, 4)	jal calculate
a, b	$a0, $a1
return result	result in $v0
doubleValue(sum)	jal double_value
doubleValue()	Leaf function
calculate()	Non-leaf function

- Given the C++ code, write the equivalent MIPS program. Your MIPS program should demonstrate the use of the stack pointer ($sp) and correctly implement the leaf and non-leaf functions discussed in clas